Dissertation Proposal

Programme: MSc in Artificial Intelligence	**Year:** 2026
Student Name: Sumukh Guruprasad	**Term:** Final / Dissertation Term
Student ID: 2443400	**Supervisor:** Mustansar Ghazanfar

Title of the dissertation:

GSIP: A Distributed Platform for Sample-Efficient Multi-Objective Optimisation of Expensive Black-Box Simulations

1. Project Summary

Many practical optimisation problems, engineering design, hyperparameter tuning, policy search, financial risk calibration depend on simulations that are expensive to evaluate and have multiple competing objectives. Two dominant approaches address this: Bayesian Optimisation (BO), which is highly sample-efficient but scales poorly with dimensionality and objective count, and Multi-Objective Evolutionary Algorithms such as NSGA-II, which handle Pareto fronts well but require thousands of evaluations. This project will design and implement a distributed simulation platform that combines BO and NSGA-II within a single search loop, with parallel evaluation orchestrated across a compute cluster. The platform will be evaluated on standard multi-objective benchmarks (ZDT, DTLZ) and compared against NSGA-II and BO baselines on convergence speed and solution quality. The intended contribution is empirical evidence on whether a hybrid Bayesian-evolutionary loop can reduce expensive evaluations on multi-objective problems without sacrificing front coverage.

2. Research Area

Real-world decision and design problems often involve simulating a system whose evaluation is computationally expensive, a finite-element model, a Monte Carlo financial simulator, or training a deep neural network with a particular hyperparameter set. When such systems are also multi-objective (for example, minimising cost and risk simultaneously), the search problem becomes one of approximating the Pareto front with as few simulators calls as possible. This problem class has been studied for two decades but remains active because most production tools force practitioners to choose between two families of methods that each have known weaknesses.

Bayesian Optimisation (Shahriari et al., 2016) builds a probabilistic surrogate, typically a Gaussian Process, over the objective and selects new points by maximising an acquisition function such as Expected Improvement. It is the canonical sample-efficient approach for expensive black-box problems and underpins production tuning systems such as Google Vizier (Golovin et al., 2017). Multi-objective extensions exist — qEHVI (Daulton et al., 2020) and ParEGO (Knowles, 2006) — but Gaussian Process inference scales cubically

with the number of observations, and acquisition optimisation becomes harder as objectives and dimensions grow. In practice, pure BO is limited to problems with a few thousand evaluations and modest dimensionality.

The evolutionary tradition takes the opposite trade-off. NSGA-II (Deb et al., 2002) and its successors (NSGA-III, MOEA/D) maintain a population, rank candidates by non-domination, and preserve diversity through crowding distance. They scale gracefully to many objectives and high dimensions and are straightforward to parallelise because the offspring of a generation are independent. Their weakness is sample efficiency: they typically need tens of thousands of evaluations, which is fatal when each evaluation is a long simulation.

Hybrid approaches have been proposed to combine these strengths. Surrogate-assisted evolutionary algorithms (Jin, 2011) train a regression model alongside the population and use it to pre-screen offspring. More recent work — for example, MOEA/D-EGO (Zhang et al., 2010) and trust-region BO (TuRBO, Eriksson et al., 2019) — attempts a tighter coupling, but most published prototypes are research code that has not been packaged for reproducible distributed execution. There is, in short, a gap between the algorithmic literature and an open, distributed implementation that practitioners can run on real expensive simulators with a clear baseline comparison.

A second strand of relevant work is distributed execution. Ray (Moritz et al., 2018) has become the de facto framework for distributed Python workloads in machine learning, with built-in support for parallel tuning via Ray Tune. Asynchronous parallel BO has been studied by Kandasamy et al. (2018) and others, who show that the surrogate must be carefully updated to avoid wasting evaluations when results return out of order. For evolutionary algorithms, parallel evaluation of an entire generation is straightforward, but synchronisation cost grows with population size.

The contribution of this dissertation is therefore not a new algorithm but an empirical investigation. I will implement a distributed platform — provisionally named GSIP — that runs NSGA-II and Bayesian Optimisation as two cooperating search policies sharing a common evaluation pool, executed in parallel via Ray. The Bayesian component will bias the evolutionary search by occasionally injecting acquisition-maximising candidates into the population, and the evolutionary component will provide diverse seed points to the BO surrogate. The platform will be benchmarked on the ZDT (Zitzler et al., 2000) and DTLZ (Deb et al., 2002) function suites — the standard testbeds in the field — using Hypervolume and Inverted Generational Distance as the principal metrics.

The research questions are: (i) does a cooperating BO/NSGA-II loop converge in fewer evaluations than either baseline on standard multi-objective benchmarks; (ii) how does the answer to (i) change as the number of objectives and the dimensionality of the search space increase; and (iii) what is the wall-clock cost of the hybrid loop relative to parallel NSGA-II when evaluations are genuinely expensive.

3. Expected Practical Element Output

The practical deliverable is an open-source Python software platform that implements the proposed hybrid Bayesian-evolutionary optimisation loop with distributed execution. The minimum viable scope of this software is:

- A core search loop wrapping two pluggable optimiser backends — NSGA-II (via pymoo) and Bayesian Optimisation (via BoTorch) — behind a single interface.

- A coordinator that schedules candidate evaluations as parallel tasks on a Ray cluster, supports both synchronous and asynchronous modes, and reconciles returned results into a shared archive of evaluated points.
- A benchmark harness that runs the platform against ZDT1–ZDT6 and DTLZ1–DTLZ4 problems with configurable budgets and seeds.
- A reporting module that exports per-run Hypervolume and Inverted Generational Distance curves, Pareto front plots, and summary tables for inclusion in the final dissertation.

The system will be containerised with Docker and runnable both locally on a laptop (single-node Ray cluster) and on a small cloud VM for higher-budget experiments. Configuration will be specified through YAML files so experiments are reproducible from a single command.

The deliverable is intentionally scoped to a minimum viable platform rather than a polished product. The goal at submission is a working system that produces the experimental data the dissertation analyses, with clear extension points. Possible extensions, pursued if time allows and otherwise noted as future work, include: replacing the in-memory archive with a vector store such as Milvus for warm-starting from prior runs; orchestrating long-running experiment campaigns with Temporal; integrating real applied simulators (an engineering or financial benchmark); and supporting many-objective methods such as NSGA-III and MOEA/D.

4. Required Resources

The project is principally software. The required resources are:

- A development laptop, which I already own, sufficient for code development, single-node Ray clusters, and small-budget benchmark runs.
- Cloud compute for the final benchmark sweep where the budget exceeds what a laptop can finish overnight. A modest budget of approximately £100 on a pay-as-you-go provider (Google Cloud, AWS, or Hetzner) is anticipated for short-lived virtual machines. No GPU is required because the benchmark problems are analytic functions rather than neural-network training.
- Open-source software only. The core dependencies — Ray, pymoo, BoTorch, NumPy, SciPy, Matplotlib, Pandas, and Docker — are free and installable via standard package managers. No proprietary licences are required.
- Access to the University library and standard academic databases (IEEE Xplore, ACM Digital Library, arXiv) for the literature component. Access is already available through my UEL credentials.
- A GitHub repository to host the code, version history, and continuous integration. A standard free account is sufficient.

No specialist equipment, no human participants, and no datasets requiring ethics approval are involved.

5. Prerequisite Knowledge / Skills Required

The project draws on four areas in which I have prior experience.

- Distributed systems and Python engineering. From founding-engineer work at NextGen Education and from building Agile-Flow — a multi-tenant SaaS platform using NestJS, Redis, Kafka and BullMQ — I am comfortable with concurrent task execution, queueing and observability. Ray is new at the project start, but its actor and task model is familiar from prior queue and workflow work.

- Numerical optimisation. My MSc in Artificial Intelligence at UEL has covered convex optimisation and gradient-based methods. Bayesian Optimisation and evolutionary multi-objective optimisation are the principal new material; I have done preliminary reading and built a small pymoo prototype while scoping this proposal.
- Machine learning and probabilistic modelling. Gaussian Processes are unfamiliar in depth; I will study Rasmussen and Williams (2006) and the BoTorch documentation during the first weeks of the project.
- Empirical research methodology. From prior work on a deep-learning skin-lesion classification study and a Bitcoin market-cycle research paper, I am comfortable with experimental design, ablation studies and writing up quantitative results.

Self-study, supervisor meetings and reproduction of existing baselines will close the remaining gaps.

6. Project Plan — Gantt Chart

The project follows a 10-week plan. Implementation and experimentation run in parallel with writing during the second half. Submission is scheduled for week 10.

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Literature review & question refinement										
Implement NSGA-II baseline (pymoo)										
Implement BO baseline (BoTorch)										
Distributed evaluation harness (Ray)										
Hybrid cooperating loop										
Benchmark integration (ZDT, DTLZ)										
Final experimental runs										
Analysis & figures										
Methods chapter draft										
Results chapter draft										
Introduction & discussion drafts										
Revisions & final formatting										
Submission										

References

Daulton, S., Balandat, M. and Bakshy, E. (2020). Differentiable expected hypervolume improvement for parallel multi-objective Bayesian optimization. NeurIPS.

Deb, K., Pratap, A., Agarwal, S. and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, 6(2), pp. 182–197.

Eriksson, D., Pearce, M., Gardner, J. R., Turner, R. and Poloczek, M. (2019). Scalable global optimization via local Bayesian optimization (TuRBO). NeurIPS.

Golovin, D., Solnik, B., Moitra, S., Kochanski, G., Karro, J. and Sculley, D. (2017). Google Vizier: A service for black-box optimization. KDD.

Jin, Y. (2011). Surrogate-assisted evolutionary computation: Recent advances and future challenges. Swarm and Evolutionary Computation, 1(2), pp. 61–70.

Kandasamy, K., Krishnamurthy, A., Schneider, J. and Póczos, B. (2018). Parallelised Bayesian optimisation via Thompson sampling. AISTATS.

Knowles, J. (2006). ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. IEEE TEC, 10(1).

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I. and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. OSDI.

Rasmussen, C. E. and Williams, C. K. I. (2006). Gaussian Processes for Machine Learning. MIT Press.

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P. and de Freitas, N. (2016). Taking the human out of the loop: A review of Bayesian optimization. Proceedings of the IEEE, 104(1), pp. 148–175.

Zhang, Q., Liu, W., Tsang, E. and Virginas, B. (2010). Expensive multiobjective optimization by MOEA/D with Gaussian process model. IEEE TEC, 14(3), pp. 456–474.

Zitzler, E., Deb, K. and Thiele, L. (2000). Comparison of multiobjective evolutionary algorithms: Empirical results. Evolutionary Computation, 8(2), pp. 173–195.